Projects in Machine Learning and AI (RPI Spring 2026)
Lab 3: Hugging Face Pipelines for NLP and Vision

Two-week lab • 2 checkpoints • Inference-first (no training required)

Overview

In this lab you will use the Hugging Face Transformers `pipeline` API to rapidly prototype NLP and vision applications using pretrained Transformer-based models. You will build, evaluate, and compare pipelines across tasks, and practice reading model cards to justify model choices.

Important: To avoid overlap with Homework 5, this lab focuses on using pretrained models for inference and lightweight evaluation. Do not implement attention/Transformer blocks from scratch or train CNNs/Transformers end-to-end in this lab.

Learning objectives

- Create and run Hugging Face pipelines for multiple NLP tasks (e.g., sentiment analysis, NER, QA, summarization).
- Create and run pipelines for vision tasks (e.g., image classification, object detection, image-to-text, zero-shot image classification).
- Evaluate pipelines on small datasets using `datasets` + `evaluate` and report basic metrics (e.g., accuracy/F1) and inference time.
- Compare two models for the same task and justify tradeoffs using evidence from model cards (intended use, limitations, license).
- Apply practical inference engineering: batching, max length, truncation, device selection (CPU/GPU).

Deliverables

You will submit two short notebooks (or one notebook with two clearly labeled sections):

- Checkpoint 1 (Week 1): NLP Pipelines Prototype + Evaluation
- Checkpoint 2 (Week 2): Vision (and optional multimodal) Pipelines Prototype + Evaluation

Each checkpoint should include: (1) runnable code, (2) a small results table, and (3) a short discussion section.

Environment setup

Recommended: Python 3.10+ in a virtual environment (venv/conda).

```
pip install -U transformers datasets evaluate accelerate torch torchvision pillow
```

If you have a CUDA GPU available, install a CUDA-enabled PyTorch build (see PyTorch installation instructions).

Starter imports

```
from transformers import pipeline
from datasets import load_dataset
import evaluate
import time
from PIL import Image
```

Scope and guardrails (to avoid overlap with Homework 5)

In scope:

- Using `pipeline` for inference with pretrained models.
- Running evaluation on small subsets (e.g., 100-500 examples) and reporting basic metrics.
- Comparing multiple pretrained models for the same task.
- Reading and citing model card content (in your own words) to justify choices.

Out of scope for this lab:

- Training CNNs from scratch, transfer learning fine-tuning, or data augmentation experiments (Homework 5 Task 1).
- Implementing VAEs or GANs (Homework 5 Task 2).
- Implementing scaled dot-product attention, seq2seq attention, or Transformers from scratch (Homework 5 Task 3).
- Machine translation training experiments (Homework 5 Task 3).

Checkpoint 1 (Week 1): NLP pipelines

Goal: build a small NLP pipeline suite, evaluate it on a dataset subset, and compare at least two models for one task.

Part A: Choose two NLP tasks

Pick exactly TWO tasks from the list below and implement each using `pipeline`:

- Sentiment analysis (text-classification)
- Topic classification via zero-shot classification (zero-shot-classification)
- Named entity recognition (ner)
- Question answering (question-answering)
- Summarization (summarization)

Part B: Dataset subset and evaluation

For at least ONE of your chosen tasks, evaluate on a small dataset subset using `datasets` + `evaluate`. Suggested datasets (choose one appropriate for your task):

- Sentiment: `imdb` or `tweet_eval` (subset only)
- NER: `conll2003` (use a small split subset)
- QA: `squad` (subset only)
- Topic/zero-shot: `ag_news` (subset only)

Report at least one metric (accuracy or F1) and also report average inference time per example. If your task output is not easily scorable (e.g., summarization), use qualitative evaluation plus runtime statistics.

Part C: Model comparison (required)

Choose ONE task and compare TWO different models for it. Your comparison must include:

- A small table: metric(s), average inference time, and model name/ID for both models
- A short justification referencing model-card information (e.g., training data domain, intended use, limitations, license)
- One concrete failure case or surprising behavior you observed

Suggested code patterns

```
# Example: sentiment analysis
clf = pipeline('text-classification', model='distilbert-base-uncased-finetuned-
sst-2-english')
clf('I loved the pacing but the ending felt rushed.')


# Timing helper
def avg_latency(fn, inputs, n=50):
    t0 = time.time()
    for i in range(n):
        fn(inputs[i])
```

```
    return (time.time() - t0) / n
```

Tip: Use batching where supported (e.g., pass a list of inputs) and record the batch size you used.

Checkpoint 1 submission checklist

- Two pipeline implementations (two different NLP tasks).
- Evaluation on a dataset subset for at least one task (metric + latency).
- Two-model comparison for one task (table + discussion + failure case).
- Clear notebook organization and runnable cells top-to-bottom.

Checkpoint 2 (Week 2): Vision (and optional multimodal) pipelines

Goal: build a small vision pipeline suite, visualize outputs, and compare at least two models on one vision task.

Part A: Choose two vision/multimodal tasks

Pick exactly TWO tasks from the list below and implement each using `pipeline`:

- Image classification (image-classification)
- Zero-shot image classification (zero-shot-image-classification)
- Object detection (object-detection)
- Image segmentation (image-segmentation) — optional if you find a suitable model
- Image captioning / image-to-text (image-to-text)

Part B: Small image set and visualization

Create a small image set of at least 20 images. You can:

- Load a Hugging Face dataset with images (recommended).
- Download a small curated set of images yourself.

Visualization requirements (include at least 5 visualized outputs):

- Object detection: draw bounding boxes and labels.
- Segmentation: overlay masks on the image.
- Classification/zero-shot: show top-k labels and scores.
- Captioning (image-to-text): show the image and generated caption.

Part C: Model comparison (required)

Choose ONE of your vision tasks and compare TWO different models. Your comparison must include:

- A small table: qualitative notes + latency; include a metric if the dataset provides labels
- A short justification using model-card information
- At least one clear example where the models differ meaningfully (good or bad)

Suggested code patterns

```
# Example: image classification
img_clf = pipeline('image-classification', model='google/vit-base-patch16-224')
img = Image.open('my_image.jpg')
img_clf(img, top_k=5)


# Example: object detection (visualize boxes)
det = pipeline('object-detection', model='facebook/detr-resnet-50')
pred = det(img)
# pred: list of dicts with 'box' (xmin, ymin, xmax, ymax), 'label', 'score'
```

Tip: When running on GPU, pass `device=0` to the pipeline constructor (if using a single GPU).

Checkpoint 2 submission checklist

- Two vision/multimodal pipeline implementations (two different tasks).
- At least 20 images processed; at least 5 outputs visualized.
- Two-model comparison for one task (table + discussion + example differences).
- Clear notebook organization and runnable cells top-to-bottom.

Submission format

Submit links to your notebook(s) (Colab/GitHub) or upload the .ipynb. If you use external images, include code that downloads them so your notebook is reproducible.

Grading rubric (100 points)

Component	What we look for	Notes
Checkpoint 1: NLP	Two tasks + evaluation + model comparison + discussion	Subset evaluation is fine (100-500 examples).
Checkpoint 2: Vision	Two tasks + visualization + model comparison + discussion	At least 20 images; show at least 5 visual outputs.
Model card reasoning	Clear justification of model choice + limitations/bias notes	Use your own words; cite model IDs.
Reproducibility	Notebook runs top-to-bottom; fixed seeds where relevant; clear environment notes	Avoid hidden/manual steps.

Optional extensions (no extra credit unless announced)

- Batching experiments: compare per-example latency for batch_size = 1 vs 8 vs 32.
- Try a domain-specific model (e.g., clinical, legal) and analyze where it helps/hurts.
- Try quantization or `torch_dtype=torch.float16` (GPU) and report speed/memory effects.
- Build a tiny demo function that wraps your best pipeline into a clean API (one function call).